

本科毕业论文答辩问题与应答指南

基于论文：《基于 eBPF 的开源 AI 推理引擎的跨语言调用链分析》(HybriChain)

靶点系统：Ollama 0.22.0 (Go-CGO-C++ 三层混合架构)

第 1 章 绪论 (10 题)

Q1: 你的研究背景是什么？为什么要研究开源 AI 推理引擎的调用链追踪？

答：

研究背景有两个层次。第一层是 AI 推理框架的系统复杂性危机：随着大模型参数规模指数级膨胀，底层硬件的内存带宽与计算能力博弈推向极端，迫使推理引擎在系统软件层进行极其复杂的工程优化，普遍采用多进程解耦与多语言混合编程 (Go/Python/C++)。第二层是安全可观测性断层：这种复杂架构使得推理引擎内部的数据流转行为对安全运维人员完全不透明，当发生恶意入侵（如目录遍历攻击、畸形载荷注入）时，运维人员既无法追溯外部数据在推理引擎内部的完整流转路径，也无法回答该行为在内核层面触发了哪些系统调用。这两个层次叠加，催生了本研究的核心需求——对多语言混合推理引擎进行跨语言调用链追踪与安全分析。

Q2: Problama 漏洞 (CVE-2024-37032) 的具体成因是什么？它如何暴露了多语言架构的安全问题？

答：

Problama 漏洞的底层成因是：Ollama 守护进程在处理高层 API 传输的模型数据摘要 (Digest) 时，未能严格验证其十六进制格式规范。具体来说，攻击者通过构造携带路径穿越符（如 `../`）的畸形 API 数据（伪装成合法的 Digest），成功绕过系统沙箱，实现了对宿主机底层任意核心文件的越权读取。该漏洞的典型意义在于：它揭示了从高层 API 接口到中层语言桥接 (CGO) 再到内核文件系统的完整数据流，在现有可观测工具下无法被连贯还原。安全运维人员事后只能通过日志推断攻击路径，但无法实时追踪数据在推理引擎内部的跨语言流转过程。这正是本文要解决的核心问题——建立从高层语义到低层内核执行的完整追踪链路。

Q3: 为什么 PyTorch TorchServe 的 CVE-2023-43654 漏洞与你的研究相关？

答：

TorchServe 的 CVE-2023-43654 漏洞与本文研究的关联在于：两者都揭示了“推理框架数据传输 API 层”的安全脆弱性。TorchServe 的漏洞成因是架构允许未经严格身份验证的远程用户通过 API 上传模型存档，在反序列化阶段触发远程代码执行 (RCE)。这说明推理框架的 API 层是一个高危攻击面——外部数据从这里进入推理引擎内部。然而，无论是 TorchServe 还是 Ollama，事后修复都停留在“打补丁”和“特定路径过滤”层面，没有建立一套自动化、实时的底层追踪链路来监控外部数据从 API 入口到内核文件读写的完整流转过程。本文正是要填补这个空白，提供通用的跨语言调用链追踪与安全分析手段。

Q4: 本文的核心研究目标是什么？用一句话概括。

答：

本文的核心研究目标是：设计并实现 HybriChain——一个面向多语言推理引擎的跨语言调用链追踪与安全分析系统，使得安全运维人员能够从高层 API 请求一路追踪到内核系统调用，完整还原推理引擎内部的数据流转路径，并自动化检测其中的异常行为。

Q5: 本文的主要创新点是什么？

答：

本文有四个主要创新点：

- 跨语言符号映射机制**：提出五层符号分类体系 (f_0 – f_4)，将来自不同命名空间（Go 运行时、CGO ABI、底层计算库）的函数符号统一映射到同一语义空间。
 - 轻量级 eBPF 探针下沉战略**：仅对推理计算核心函数挂载 uprobe，将运行时开销降至最低（TTFT 增量 < 6.3%），避免了传统 DBI 的性能灾难。
 - 动静结合的调用链缝合算法**：以纳秒级高精度时间戳为纽带，将静态符号图谱与动态 eBPF 事件轨迹融合，每条跨语言边赋予 Confirmed/Inferred/Unknown 置信状态。
 - 异常行为分析引擎**：支持对 futex 阻塞、异常 mmap 分配、敏感路径访问等高危行为的自动化检测。
-

Q6: 为什么现有工具（strace、GDB、DynamoRIO、Valgrind）无法解决你的问题？

答：

各工具的局限性如下：

- strace**：基于 ptrace 用户态拦截，每条系统调用产生两次上下文切换，高频调用场景下性能开销极高，且无法获取用户态函数调用信息。
 - GDB**：需要调试信息，交互式调试工具，无法自动化追踪。
 - DynamoRIO**：虽然轻量，但需要在目标进程内注入运行，基于编译器后端拦截，难以实施非侵入式的三方安全审计，且 ABI 与 Go 1.17+ 不兼容。
 - Valgrind/Memcheck**：最精确但最沉重——构建影子内存和指令级翻译导致数十倍性能惩罚，对每秒处理数百兆张量权重吞吐的大语言模型推理引擎，会导致系统资源耗尽并崩溃。
 - eBPF**（本文方案）：在内核安全沙箱运行，由 Verifier 提供运行时安全验证，极低开销，零侵入，即时生效热更新，完美契合推理引擎的极致性能要求。
-

Q7: 你的研究与 ProfInfer 的区别是什么？

答：

ProfInfer 利用 uprobe 结合硬件性能监控计数器（PMC），在 Token 生成解码粒度上精确追踪底层张量算子（如矩阵乘法）导致的 L3 缓存未命中等微观硬件物理指标。但 ProfInfer 的致命缺陷在于：它完全放弃了对顶层数据传输 HTTP API 与底层 C++ 计算之间跨语言反序列化语义的缝合。当攻击者发起应用层 API 攻击（如目录遍历、畸形载荷注入）时，ProfInfer 无法提供任何溯源保护——它只告诉你哪个硬件指标出了问题，但不告诉你这个问题是由哪个 API 请求触发的、从哪条调用链传导而来。HybriChain 的核心差异化价值在于：从 HTTP API 层一路追踪到内核 syscall，完整还原跨语言数据流。

Q8: 为什么 llama.cpp 要用纯 C/C++ 实现？这与你的研究有什么关系？

答：

llama.cpp 用纯 C/C++ 实现是为了追求极致的底层控制与跨平台部署能力——它通过深度整数级混合量化（Quantization）技术和全平台指令集优化，解决了在缺乏专用 GPU 的边缘设备与消费级处理器上部署千亿级参数大模型的难题。但这种极致底层的纯 C/C++ 控制，导致其难以直接且安全地对接现代基于 Go 或 Python 的高并发网络微服务 API 层，从而衍生出 Ollama 这样跨越 Go → CGO → C++ 三层语言的复杂封装层。这三层语言的跨越，正是 HybriChain 需要追踪的核心对象——没有这种跨语言架构，就不需要 HybriChain。

Q9: 你的论文结构是如何安排的？每章之间是什么关系？

答：

本文六章形成一个完整的“发现问题 → 分析问题 → 设计方案 → 工程实现 → 实验验证 → 总结展望”的闭环：

- **第1章（绪论）**：发现并定义问题——多语言混合编程架构带来了可观测性断层，以及研究它的紧迫性和意义。
- **第2章（相关技术综述）**：穷尽校验 30 项重磅文献，论证现有技术（eBPF、静态分析、DBI）各有局限，明确本文方法的唯一性与不可替代性。
- **第3章（系统设计）**：提出 HybriChain 的整体架构与四大核心模块（追踪 → 降噪 → 缝合 → 映射）。
- **第4章（实现方案）**：以 Ollama 为靶点，将第3章的设计方案一一落地为具体代码实现。
- **第5章（实验评估）**：用数据证明 HybriChain 的有效性（准确性、低开销、异常检测能力）。
- **第6章（总结与展望）**：总结贡献，坦诚局限，展望未来工作。

Q10: 你在论文中引用了 2026 年的文献，这合理吗？这些文献是真实存在的吗？

答：

（坦诚回答）需要说明的是，论文中引用的部分文献年份标注为 2025 年甚至 2026 年，这是为了论证的完整性而作的合理推演或对趋势性工作的引用。具体到答辩时，建议将这类文献替换为真实可查的已发表文献，或与导师确认其真实性。如果答辩老师追问，我应当坦诚说明哪些是趋势性引用、哪些是真实文献，并承诺在最终版本中核实所有引用。

第 2 章 相关技术综述（15 题）

Q11: eBPF 的 Ring Buffer 相比 Perf Buffer 解决了什么问题？

答：

传统 Perf Buffer 在多核高并发大流量场景下存在两个严重问题：

1. **事件拥塞丢失**：Perf Buffer 是单生产者单消费者（SPSC）模型，多核场景下多个 CPU 同时向 buffer 写入会产生竞争，导致高频事件数据丢失。
2. **时间戳乱序（Event Re-ordering）**：不同核的事件写到同一个 buffer 时，失去原本的时间顺序。

A. Nakryiko 引入的 BPF Ring Buffer 通过构建多生产者单消费者（MPSC）共享内存队列，彻底解决了这两个问题：它允许无锁的并发写入（多核并行），同时保证 FIFO 顺序，是高频追踪场景下数据回传的最佳选择。HybriChain 正是利用 Ring Buffer 接收 uprobe 和 tracepoint 事件，确保不丢失跨语言调用链的关键事件。

Q12: 为什么 Go 1.17+ 与传统 DBI 工具（如 Pin）ABI 不兼容？

答：

Go 1.17 引入了一场重大的 ABI 变革：从基于堆栈的调用约定（stack-based calling convention）切换到基于寄存器的调用约定（register-based calling convention，类似于 System V AMD64 ABI）。这一改变使得 Go 函数的参数传递方式与传统 DBI 插桩工具的预期完全不同——Pin 等 DBI 工具假设函数参数在栈上，而 Go 1.17+ 函数的参数部分或全部放在寄存器中。这导致插桩工具无法正确读取或修改函数参数，极端情况下甚至会导致进程崩溃。此外，Go 的垃圾回收器（GC）引入了复杂的栈展开（stack unwinding）行为，进一步增加了 DBI 插桩的不确定性。因此，HybriChain 摒弃 DBI，全面转向基于 eBPF 的追踪方案。

Q13: CGO 哈希后缀（`_cgoexp_HASH_`）是如何产生的？为什么它导致符号不可读？

答：

当 Go 程序中使用 CGO 调用 C/C++ 代码时，Go 编译器会在编译期自动生成桥接函数。这些桥接函数名原本应该对应原始的 C 函数名，但 Go 编译器对其进行了哈希处理，生成形如 `_cgoexp_HASH_原函数名` 的格式，其中 HASH 是基于包路径和函数签名的 SHA256 摘要的前缀。这种设计的目的是避免命名冲突和确保链接时的确定性。但其副作用是：生成的哈希后缀无法通过标准的 `nm` 工具直接还原原始函数签名，标准的 demangling 工具也无法处理，导致安全分析人员无法直观理解某次跨语言调用对应的是哪个原始 C 函数。HybriChain 的 `classify_func.py` 通过识别 `_cgo_` 前缀特征，将其统一分类为胶水层（ f_L ），绕过了哈希后缀的障碍。

Q14: POLYCALL 的降维思路是什么？它为什么无法处理静态链接的推理引擎？

答：

POLYCALL（中科大安全实验室提出）的核心降维思路是：将所有高级语言统一下沉编译至 WebAssembly（Wasm）中间表示，利用 Wasm 的统一函数索引体系来建立跨语言调用关系，解决了语法差异巨大无法统一函数索引的难题。但这个方法的严重局限在于：它依赖于动态链接库中暴露的标准导出符号表。而 llama.cpp 等追求极致效率的推理框架采用静态链接（Static Linking）将 Go 与 C++ 打包为单一二进制文件，所有符号内联，外部无法看到标准的导出符号表。因此 POLYCALL 在面对 Ollama 时完全失效。HybriChain 的方法不需要修改编译方式，在运行时通过 eBPF 直接追踪实际执行的函数，天然兼容静态链接。

Q15: 静态污点分析与动态追踪的根本区别是什么？为什么静态分析无法解决你的问题？

答：

静态污点分析（如 Brucker 团队的工作、CHARON 框架、Deepwukong 图神经网络方法）本质上是回答“程序可能如何运行”——它通过源码或二进制分析，推导所有可能的数据流路径，生成一个庞大的潜在调用图。其根本局限在于：

1. **静态图谱缺乏时间维度**：无法告诉你某个 API 请求在运行时实际触发的是哪条路径。
2. **无法捕获动态行为**：面对操作系统并发竞态、系统调用劫持、通过 `mmap` 发起的动态巨页内存分配时，静态分析完全失效。
3. **面对哈希加扰无能为力**：Go 编译器故意注入的随机哈希后缀使得静态符号解析充满不确定性。

相比之下，HybriChain 的动态 eBPF 追踪回答的是“程序实际如何运行”——它在运行时捕获真实的函数调用事件，提供时间戳、线程上下文、调用栈等运行时信息，是静态分析的有力补充而非替代。

Q16: 五层符号分类体系 (f_0 – f_4) 中的 f_3 (内存管理层) 与 f_0 (系统边界层) 有什么区别？

答：

f_3 (内存管理层) 和 f_0 (系统边界层) 都涉及内存操作，但层次和视角不同：

- **f_3 (内存管理层)** 是用户态视角，关注推理引擎内部如何管理 KV Cache、模型权重加载、上下文管理，代表性函数如 `ggml_backend_buffer_*`、`llama_model_load_from_file`，这些是推理引擎的逻辑内存管理行为。
- **f_0 (系统边界层)** 是内核视角，关注推理引擎向操作系统发起的系统调用，如 `mmap` / `brk` / `mprotect`，这些是内核层面的物理内存分配行为。

两者的关系是： f_3 层函数在执行过程中会触发 f_0 层的系统调用。例如，`llama_model_load_from_file` (f_3) 在内部会调用 `mmap` (f_0) 将权重文件映射到内存。因此， $f_3 \rightarrow f_0$ 的跨层调用关系，意味着“推理引擎的内存管理层触发了内核的系统内存分配调用”，这是 HybriChain 缝合算法的重要语义线索。

Q17: 为什么 PagedAttention 机制缺乏安全隔离探讨会影响你的研究？

答：

PagedAttention 利用操作系统虚拟内存的按需分页思想管理 KV Cache，实现了吞吐量的几何级攀升，这是纯性能优化。但本文指出它“极度优化了内存复用率，却缺乏对跨并发请求在混合内存块池中相互越界读写的深入安全隔离探讨”。这意味着在高并发场景下，不同用户的请求可能共享同一块内存池，如果 HybriChain 能够追踪跨请求的内存分配调用链 ($f_3 \rightarrow f_0$ 的 `mmap`)，理论上可以检测出这类越界访问。这为 HybriChain 的安全分析能力提供了一个扩展方向——这也是论文第6章未来工作可以延伸的点。

Q18: eBPF Verifier 的作用是什么？它为什么能保证追踪的安全性？

答：

eBPF Verifier 是 eBPF 架构安全的核心保障。当用户提交一个 BPF 程序到内核时，Verifier 在程序加载阶段执行严格的安全检查：

1. **穷尽性检查**：确保程序中不存在任何可能导致内核崩溃的路径（如空指针解引用、越界内存访问）。
2. **资源边界检查**：确保程序不会消耗超过限制的栈空间（最多 512 字节）和循环次数（有界循环检查）。
3. **类型检查**：确保所有内存访问都是类型安全的。

通过这三层检查，eBPF 程序在运行前就被“数学证明”为安全的，用户态注入恶意 BPF 程序无法导致内核崩溃或提权。这使得 eBPF 可以在不修改目标进程的前提下，以最小权限在内核中安全执行观测逻辑。相比之下，`strace` 和 `GDB` 完全缺乏这类安全验证机制。

Q19: cilium/ebpf 库解决了什么问题？为什么它对 HybriChain 重要？

答：

cilium/ebpf 是 Cilium 团队开源的纯 Go 语言 eBPF 交互库，解决了云原生环境中 Go 语言无法高效、原生地下发与解析 eBPF 字节码的障碍。在此之前，Go 程序与 eBPF 交互需要通过外部脚本（如 bpftrace）或复杂的 CGO 桥接，引入额外的复杂性和性能开销。cilium/ebpf 允许纯 Go 程序直接编译、加载、挂载和管理 eBPF 程序，是 HybriChain 技术栈的核心基础设施——HybriChain 的 Python 后处理脚本（trace_unify.py、stitcher.py 等）之上，实际运行的 eBPF 程序就是通过类似机制部署的。

Q20: 图神经网络方法（Deepwukong）在跨语言数据流分析中的优势和局限是什么？

答：

Deepwukong 的优势在于：它利用图神经网络（GNN）自主学习漏洞在调用图结构中的拓扑分布模式，解决了依赖人工编写繁杂静态正则表达式检测未知变种漏洞的瓶颈——它能发现传统静态规则无法覆盖的深层调用路径关联。

但其局限同样明显：图神经网络训练依赖大量标注数据，且模型的泛化能力受限于训练集的覆盖度；在面对推理引擎的实时动态调用时，训练好的模型可能无法准确识别未见过的调用模式；此外，GNN 的推理开销对于在线实时检测也是一个挑战。HybriChain 采用规则引擎（而非 ML 模型）进行异常检测，虽然泛化能力有限，但检测逻辑透明、可解释，符合安全审计的取证标准。

Q21: Go-CGO 互操作有哪些已被证实的类型安全缺陷（J. Chen 等人的研究）？

答：

J. Chen 等人对海量开源项目进行了实证扫描，总结出 19 种特定于 CGO 的致命缺陷模式，其中最典型的几类包括：

1. **指针逃逸（Pointer Escape）**：Go 的垃圾回收器无法追踪通过 CGO 传递给 C 代码的指针，当 C 代码持有该指针并在其生命周期外使用时，GC 可能将其错误回收，导致 use-after-free。
2. **类型混淆（Type Confusion）**：Go 和 C 的类型系统在内存布局上存在差异（如结构体对齐），CGO 传递复杂数据类型时容易出现隐式类型转换错误。
3. **内存生命周期管理冲突**：Go 的 GC 和 C 的手动内存管理对同一块内存的生命周期认知不一致，导致双重释放或泄漏。
4. **并发安全（Thread Safety）**：Go 的 goroutine 调度器与 C 代码的单线程假设冲突，在 C 代码执行期间触发 GC 可能导致数据竞争。

这些缺陷是 HybriChain 异常检测规则设计的现实依据——例如，异常的内存分配行为可能正是指针逃逸的前兆。

Q22: 为什么不直接在 Ollama 源码中加日志来追踪调用链？

答：

直接在 Ollama 源码中加日志有三个根本性问题：

1. **需要修改源码**：Ollama 是一个外部开源项目，在生产环境中不能随意修改其代码，否则会破坏版本管理、引入兼容性问题，且每次版本更新都需要重新打补丁。

2. **性能影响**：在每个函数入口/出口插入日志语句会累积可观的运行时开销，尤其在高频调用的推理核心路径上（如 `llama_decode` 每秒执行数万次），日志开销可能超过推理本身。
3. **无法追踪内核层**：日志只能覆盖用户态函数调用，无法直接观测系统调用（`syscall`），而推理引擎的内存分配、文件读写等关键行为都要通过内核完成。

eBPF 的核心优势恰恰在于：**零侵入**——不需要修改目标程序，不需要重新编译，即插即用，同时覆盖用户态和内核态。

Q23: 动态二进制插桩（DBI）体系中，DynamoRIO 和 Valgrind 的设计哲学有何本质差异？

答：

DynamoRIO 和 Valgrind 代表了 DBI 领域的两种截然不同的设计哲学：

- **DynamoRIO**：追求“绝对透明与轻量”。其核心机制是构建一个软件“代码缓存（Code Cache）”沙箱，将原始程序的控制流重定向到缓存中执行，在缓存中插入插桩代码。由于重定向发生在运行时缓存层面，避免了频繁的内核态-用户态上下文切换，因此在某些场景下甚至能略微提升性能（通过代码优化）。但其代价是插桩粒度相对较粗。
- **Valgrind/Memcheck**：追求“极端精确”。它在指令级别通过 JIT 虚拟机解释执行原始程序，为每个内存比特维护平行的“影子状态”（Shadow Values），能够精确追踪每一比特内存的初始化状态和使用合法性。这种精确度是无与伦比的，但代价是 10–100 倍的性能惩罚。

HybriChain 选择了介于两者之间的第三条路：eBPF 的 `uprobe` 在函数级别插入极细粒度的观测点，同时通过 Verifier 保证安全性，通过 Ring Buffer 保证低开销，是精度与性能的最佳折中。

Q24: Go 的反射（reflect）包为什么对静态调用链分析构成挑战？

答：

Go 的 `reflect` 包允许程序在运行时动态地获取变量的类型信息并通过接口调用方法，调用目标在编译期完全未知。例如，`reflect.ValueOf(x).MethodByName("DoSomething").Call(...)` 的实际调用目标要到运行时才能确定，静态分析无法预测。这种动态方法分发使得静态符号表无法穷尽所有可能的调用边——即使知道调用源，也无法确定调用目标。`llama.cpp` 虽然是纯 C++，但 Ollama 的 Go 层使用反射来动态加载插件和调用方法。HybriChain 的解决方案是：对于反射调用，只能依赖动态追踪（`uprobe` 捕获实际执行的方法），静态图谱中将这类边标记为 `Unknown`，由动态追踪补充。

Q25: 为什么 ProfInfer 无法追踪 HTTP API 层到 C++ 层的跨语言语义？

答：

ProfInfer 的设计目标是从硬件性能计数器（PMC）的角度分析 LLM 算子级的性能瓶颈，其观测粒度在 CUDA kernel 层面（张量乘法、内存带宽等），完全放弃了上层的语言语义。它假设的调用链起点是“某次推理计算”，而不是“某次 HTTP API 请求”。因此，当攻击者发起一个恶意 API 请求时，ProfInfer 无法回答：这个请求经过了哪些 Go 函数？触发了哪些 CGO 桥接函数？最终在哪个 C++ 函数层面导致了异常行为？HybriChain 的跨语言符号映射和缝合算法，正是为了填补从 API 语义层到 C++ 算子层之间的语义断层。

第 3 章 系统设计 (25 题)

Q26: 你的系统设计遵循什么样的论证主线? "为了xxx所以做xxx"的逻辑是什么?

答:

HybriChain 的设计主线是环环相扣的四个子问题, 每一节对应一个:

1. 为了获得跨语言调用链, 首先需要高效且完备的函数调用追踪 (§3.3) → 提出探针下沉策略。
2. 为了让原始数据可用, 需要进行高效统一的调用链合成与降噪 (§3.4) → 提出五步漏斗式降噪流水线 + 置信度缝合算法。
3. 为了让缝合结果可解释, 需要建立准确的跨语言符号映射 (§3.5) → 提出五层符号分类体系 (f_0-f_4)。
4. 在上述成果基础上, 设计异常检测规则引擎 (§3.6) → 规则驱动的安全分析。

四者缺一不可: 追踪提供原始数据, 降噪合成让数据可用, 符号映射让结果可解释, 规则引擎实现安全分析。

Q27: 五步漏斗式降噪流水线中, "步骤三: 类型过滤"过滤了哪些系统调用? 为什么过滤它们?

答:

步骤三对系统调用类型进行白名单过滤, 仅保留高频语义事件: read/write (文件与 socket I/O)、futex (线程同步与阻塞等待)、mmap/munmap (内存分配释放)、clone3 (进程/线程创建)、openat/close (文件打开关闭)。

被过滤掉的高频噪声调用包括:

- **brk**: 堆内存微调, 在推理过程中被频繁调用数万次, 但每次调整量极小 (几 KB), 无语义分析价值。
- **mprotect**: 内存保护属性调整, 频繁但与推理语义无关。
- **madvise**: 内存管理建议调用, 同样高频但语义弱。
- **munmap**: 通常成对出现, 保留 mmap 时 munmap 必然伴随, 无需重复追踪。

这些调用在原始事件流中占 50–65%, 但对推理调用链分析无语义价值。过滤后, futex、mmap、openat 等高语义事件的计数完全不变, 降噪操作不影响后续的异常检测准确性。

Q28: 置信度缝合算法中, Confirmed、Inferred、Unknown 三种状态分别代表什么?

答:

- **Confirmed (确认)**: 通过调用栈上下文**直接确认**的跨语言调用关系。例如, uprobe 中捕获的调用栈明确包含 CGO 桥接函数 `_cgo_*_Cfunc_llama_*`, 表明 Go 层直接调用了 llama.cpp 层。这是最可靠的调用关系, 在 Ground Truth 基准上精确率达到 100%。
- **Inferred (推断)**: 通过时间窗口启发式**推断**的调用关系。例如, mmap 系统调用在某次推理函数调用之前某个时间窗口内触发, 且内存大小符合 KV Cache 分配特征 (数百 MB), 则推断两者存在关联。这种推断可能存在误匹配, 但能大幅提升召回率。

- **Unknown (未知)**: 静态符号存在于调用图中，但动态追踪未观测到实际调用关系——说明该调用路径未被本次推理请求触发，或者需要更长的采样时间。

注意：还有第四种状态 **Conflicting (冲突)**，表示同一对节点存在多个时间窗口相互矛盾的推断，说明该边可能不稳定。

Q29: 为什么仅靠 uprobe 无法直接缝合出完整的跨语言调用链?

答:

uprobe 只能观测用户态函数的入口和返回点，对于推理引擎内部大量通过系统调用与内核交互的行为（如内存分配、文件读写、线程同步），uprobe 完全无法感知——这些行为发生在内核空间，不经过用户态函数入口。此外，uprobe 捕获的调用栈只能展示用户态的调用链，Go 栈和 C++ 栈之间通过 CGO 桥接，但 CGO 内部细节在调用栈中可能缺失。因此，必须同时部署内核 tracepoint（追踪系统调用）来补充内核层的观测数据，再通过时间戳对齐将两类事件缝合为统一序列。这就是“跨语言”调用链的含义——跨越用户态（CGO/Go/C++）和内核态（syscall）的边界。

Q30: 五层符号分类体系如何处理 llama.cpp 中名字不符合规范的函数?

答:

五层分类体系采用层级降级的兜底策略:

1. **首先做精确匹配**: 对于 `llama_*`、`ggml_*`、`common_*` 等有规律前缀的函数，通过关键词分析（`batch/sampler/token` → f_2 ; `sched/graph_compute` → f_4 ; `vocab/backend` → f_3 ）进行精确分类。
2. **前两类无法覆盖的函数**: 默认归入 `llama_api`（默认兜底层），代表 llama.cpp 的通用 API 层。
3. **CGO 桥接函数**: 专门识别 `_cgo_` 前缀（ f_1 ），即使哈希后缀不可读，前缀特征也已足够将其识别为胶水层。
4. **仍无法分类的**: 在缝合结果中显示原始符号名，供人工研判。

这个策略保证了分类的覆盖度，同时通过层次标签保留了语义可解释性。

Q31: 探针下沉策略中，“仅对推理计算相关函数挂载 uprobe”具体指的是哪些函数?

答:

具体挂载的四个核心函数是:

1. `llama_decode`: 推理主循环入口，代表一次 token 生成的核心计算触发点。
2. `ggml_backend_sched_graph_compute_async`: 张量计算调度入口，代表底层算子执行的触发点。
3. `llama_synchronize`: 同步点，代表推理阶段等待前序操作完成的等待节点。
4. `common_sampler_csampler`: token 采样函数，代表推理输出生成的关键节点。

这四个函数覆盖了推理引擎从计算触发（decode）到底层执行（ggml）再到输出生成（sampler）的完整路径，是跨语言边界最密集出现的关键节点。非核心函数（如日志函数、初始化函数）不被挂载，避免引入不必要的追踪开销。

Q32: 为什么 uprobe 的 enter probe 和 return probe 需要配合使用? 只用其中一个可以吗?

答:

enter probe (uprobe) 和 return probe (uretprobe) 需要配合使用的原因:

1. **uprobe 记录进入时刻:** `@start[tid] = nsecs` 在函数入口记录当前线程的进入时间戳, 存储在 per-thread 的 BPF 哈希表中。
2. **uretprobe 计算耗时差值:** 在函数返回时, 读取 `@start[tid]` 并用当前时间戳减去进入时刻, 得到函数执行耗时 `dur = (nsecs - @start[tid]) / 1000`。
3. **uretprobe 输出完整事件:** 函数名、时间戳、线程ID、耗时及调用栈, 只有在函数返回时才完整。

如果只用 uprobe, 只能记录函数被调用了, 但不知道调用耗时, 无法分析性能瓶颈; 如果只用 uretprobe, 无法获取函数的执行时长。两者配合才能输出完整的函数调用事件 (函数名 + 进入时间戳 + 耗时 + 调用栈)。

Q33: 漏斗式降噪的步骤五"栈帧清洗"具体做了什么?

答:

步骤五"栈帧清洗"处理 uprobe 捕获的调用栈中的原始内存地址, 将其转换为可读的函数符号名:

1. **过滤内存地址帧:** bpftrace 输出中包含大量以 `0x` 开头的十六进制内存地址帧, 这些是栈上尚未符号化的返回地址, 直接过滤。
2. **保留函数符号名帧:** 保留可读的函数符号名帧, 这些才是有分析价值的函数调用关系。
3. **C++ demangle 处理:** C++ 函数名在编译期经过名字重整 (Name Mangling), 例如 `_Z13llama_decodeP14llama_context`, demangle 后还原为 `llama_decode(llama_context*)`, 使其可读。

Q34: eBPF 追踪的"零侵入"具体体现在哪些方面?

答:

"零侵入"体现在三个层面:

1. **不需要修改目标程序源码:** eBPF 探针通过动态挂载到二进制文件的函数入口/返回点实现, 不需要在源码中添加任何追踪代码, 不需要重新编译 Ollama。
2. **不需要重新部署目标程序:** eBPF 程序通过 `perf_event_open` 或 `BPF_PROG_ATTACH` 动态附加到运行中的进程, 可以热更新 (运行时修改追踪逻辑而不中断进程)。
3. **不需要符号表完整:** eBPF 探针通过函数偏移量挂载, 不强依赖 debuginfo, 即使在生产环境中也能工作 (仅需目标函数有导出符号或可预测的内存布局)。

Q35: 为什么在缝合阶段需要"按 TID 构建请求树"? 不按 TID 行不行?

答:

"按 TID 构建请求树"是缝合算法的核心前提, 不按 TID 无法正确缝合。原因在于:

1. **线程是调用关系的最小单元:** 同一个 TID 内的所有函数调用和系统调用共享同一个调用栈, 它们之间存在确定的父子调用关系。跨 TID 的事件之间不存在直接的调用关系 (除非通过 clone3/fork 创建新进程/线程)。

2. **请求树根节点的识别**：HTTP 请求在 Ollama 中通过 clone3 系统调用创建 worker 线程，以该 clone3 事件为树根节点，所有后续的 llama_decode、ggml_backend_sched_*、mmap、futex 等事件都是该请求的子树节点。
3. **置信度计算的依据**：如果两个函数符号在 ≥ 2 棵不同的请求树中共同出现，则证明它们之间的调用关系是跨请求普适的，而非偶然巧合——这就是 Confirmed 的定义基础。

如果不按 TID 构建请求树，就无法区分不同请求的生命周期，所有事件混为一谈，置信度计算完全失效。

Q36: 五层符号分类体系中的 f₄（调度协调层）有什么特征？为什么它与 futex 系统调用高度相关？

答：

f₄（调度协调层）负责推理引擎的任务分发、线程池管理和锁与同步操作，其特征是：

1. **高频 futex 系统调用**：调度线程在等待工作分配时会调用 `futex(wait)`，获得工作后调用 `futex(wake)`，这是典型的锁等待/通知模式。
2. **锁竞争行为**：当多个推理请求并发执行时，调度层是锁竞争最激烈的位置，表现为高频的 futex 系统调用序列。
3. **并发瓶颈所在**：f₄ 层的行为直接决定了推理引擎的并发吞吐量，是性能优化的关键层。

在实验数据中，Qwen2-0.5B 的 23.83 秒追踪期间共发现 30 条 HIGH 级别 futex 阻塞告警，最长阻塞 15.2 秒，这正是 f₄ 层锁竞争导致的可用性风险——调度线程持锁时间过长，导致其他线程在用户态陷入不可中断等待。

Q37: 异常检测规则引擎的三步工作流程是什么？它与缝合算法是什么关系？

答：

规则引擎的三步工作流程：

1. **遍历**：遍历缝合后的统一调用序列中的每条事件，根据事件类型（uprobe/syscall）、函数名、时间戳等字段做模式匹配。
2. **判断**：检查系统调用耗时是否超过阈值（如 `futex > 5s` → HIGH）、文件路径是否落在敏感目录（如 `/etc/passwd`）、内存分配大小是否偏离正常范围（如 `mmap > 1GB`）。
3. **告警**：条件成立时生成结构化告警，包含时间戳、线程ID、告警类型、风险等级（LOW/MEDIUM/HIGH）及详细上下文。

与缝合算法的关系：规则引擎**依赖**缝合算法的输出。缝合算法将原始的异构事件流（uprobe + syscall）融合为统一调用序列，并标注了每条边的置信状态。规则引擎在此基础上执行检测——如果直接对原始事件流执行规则，会因为噪声过多导致大量误报；只有基于缝合后的高纯度事件序列，规则引擎才能准确识别真正的异常行为。

Q38: 为什么系统目标是"统一的调用链缝合"而不是"直接追踪完整的调用链"?

答:

因为在多语言混合架构下, "直接追踪完整的调用链"在技术上不可行:

1. **Go 运行时对 uprobe 的兼容性问题:** Go 1.17+ 的寄存器调用约定使得在 Go 函数上挂载 uprobe 时, 读取函数参数变得不可靠。
2. **内核事件无法直接与用户态事件关联:** syscall 是内核态事件, uprobe 是用户态事件, 两者不在同一抽象层次, 无法直接拼接。
3. **动态链接和静态链接混用:** Ollama 的 Go 层是静态链接, C++ 层是动态链接, 统一的追踪机制无法同时覆盖两者。

因此, HybriChain 采用了"分而治之"的策略: uprobe 负责用户态函数调用, tracepoint 负责内核态系统调用, 分别处理后再通过时间戳缝合为统一序列。这不是绕路, 而是技术约束下的最优路径。

Q39: 系统的"插件化分析框架"具体指什么? 它如何体现可复用性?

答:

"插件化分析框架"体现在两个层面:

1. **靶点可替换:** HybriChain 的核心设计不依赖 Ollama 的专有接口, 而是基于通用的二进制追踪技术 (eBPF uprobe/tracepoint) 和符号分类策略。只要推理引擎采用类似的 Go-CGO-C++ 混合编程模式 (如 text-generation-webui、LocalAI), 理论上都可以复用同一套追踪框架, 只需调整探针挂载的目标函数名。
2. **规则可扩展:** 异常检测规则引擎 (analyzer.py) 以规则列表为核心, 只需新增规则条目 (定义事件类型、阈值、告警级别) 即可扩展检测能力, 无需修改核心追踪逻辑。

这使得 HybriChain 的核心代码可被多个 AI 软件复用, 而非仅为 Ollama 量身定制。

Q40: 为什么不追踪所有函数调用, 而是只追踪"关键咽喉节点"?

答:

这是精确率和召回率之间的权衡:

1. **全量追踪数据量爆炸:** 推理引擎每秒可产生数万次函数调用和系统调用。全量追踪意味着存储和计算资源迅速耗尽。
2. **关键咽喉节点足够代表调用链:** 跨语言调用链的"桥梁"集中在少数关键节点上——CGO 桥接函数 (`_cgo_*_cfunc_*`)、推理主循环入口 (`llama_decode`)、张量计算入口 (`ggml_backend_sched_*`)。追踪这些节点就能覆盖跨语言边界的主要流量。
3. **保守挂载原则:** 这是系统设计中的明确原则 (§3.3: "仅对推理计算相关函数挂载 uprobe"), 通过保守策略控制开销, 换取可接受的观测精度。

这也解释了为什么召回率只有 5% (Confirmed) 和 40% (含 Inferred) ——不是因为算法不好, 而是因为探针只挂了 4 个函数, 这是有意为之的设计选择。

Q41: 五步漏斗式降噪的压缩效果是 2–3 个数量级，具体数据是什么？

答：

实验数据：

- **TinyLlama-1.1B**：原始 25,329 行事件记录，清洗后 8,891 个有效事件，压缩比约 **2.8** 倍。
- **Qwen2-0.5B**：原始 13,670 行，清洗后 6,373 个有效事件，压缩比约 **2.1** 倍。

压缩的核心来源是步骤三（系统调用类型过滤）去掉了 `brk`、`mprotect`、`madvise`、`munmap`、`clock_gettime` 等无语义价值的内存管理调用（合计约 16,438 条，占原始输出的 50–65%）。注意：论文中提到的“2–3 个数量级”是表述上的概指（从万级压缩到千级，约 1 个数量级），实际实验数据显示是 2–3 倍的线性压缩，答辩时需注意数据的一致性。

Q42: 调用栈深度达到“数十层”意味着什么？这对缝合算法有什么影响？

答：

“数十层”意味着推理引擎的调用栈从 Go HTTP Handler 一路延伸到 `llama.cpp` 的张量计算 kernel，其间经历：Go 运行时调度器 → HTTP 路由处理 → Ollama API Handler → CGO 桥接函数 → `llama.cpp` 主循环 → ggml 张量调度 → ggml 操作实现 → CPU 指令层（SIMD）。

这对缝合算法的影响：

1. **调用栈符号化困难**：中间的 CGO 桥接层（`_cgoexp_HASH_*`）无法 demangle，导致栈帧不连续，跨语言边界的确认变得困难。
2. **间接调用求解不完整**：C++ 虚函数的动态分派和 Go reflect 的动态方法调用，使得调用栈无法完全展开，缝合算法只能捕获直接观测到的边，无法补充未观测到的间接调用。
3. **长调用链的缝合窗口设计**：对于数十层的调用链，需要设计足够大的时间窗口来匹配相邻层的调用关系，窗口太大容易产生误匹配，窗口太小容易漏掉真正的关联事件。

Q43: Confirmed 边为什么全部落在 CGO 桥接层（Go → llama.cpp）？

答：

这是 `uprobe` 探针挂载位置的直接体现。系统在 `llama.cpp` 层的四个核心函数（`llama_decode`、`ggml_backend_sched*`、`llama_synchronize`、`common_sampler_csample`）上挂载了 `uprobe`，因此只有当调用栈经过这些函数时，`uprobe` 才会记录事件。调用栈中恰好经过这些函数的跨语言调用关系才能被直接确认——而这些调用正是从 Go 层经 CGO 桥接调用 `llama.cpp` 的部分，也就是 CGO 桥接层。`llama.cpp` 内部的子函数调用（从 `llama_decode` 到 `ggml*` 的后续函数）虽然确实存在，但在这些函数上没有独立挂载 `uprobe`，因此只能通过时间窗口推断（Inferred），无法直接确认（Confirmed）。

Q44: 为什么“Unknown”边数（38 条）远多于 Confirmed 边数（3 条）？

答：

“Unknown”边来自静态符号提取阶段得到的调用图，在动态追踪中却从未被触发，原因有三：

1. **探针覆盖率有限**：当前仅在 4 个 `llama.cpp` 函数上挂载 `uprobe`，这 4 个函数内部的子函数调用在静态图谱中存在，但在动态追踪中无法被独立确认，只能标记为 Unknown。

2. **采样窗口不足**：当前 23.83 秒的 PoC 采样时长是验证框架可行性的切片，长尾路径在短时间内未被触发属于采样窗口的自然限制（如模型加载路径只在首次加载时执行）。
3. **静态图谱覆盖"所有可能路径"**：静态符号提取得到 52 条边（包括静态链接后可能内联的函数），但实际一次推理请求只走其中子集，其余边在本次请求中自然不会出现。

这说明 Unknown 并不代表系统有问题，而是"未被本次请求触发的潜在路径"，是静态与动态互补的正常现象。

Q45: "动静结合"具体指什么？静态分析和动态追踪各贡献了什么？

答：

"动静结合"的含义：

1. **静态分析贡献**：通过解析 Ollama 二进制文件的符号表（`nm -gC --defined-only`）和读取 `/proc/pid/maps`，提取出推理引擎的全局函数符号集和调用关系，构建跨语言调用图的节点集（覆盖全面但不含时序关系和运行时状态）。
2. **动态追踪贡献**：通过 eBPF uprobe/tracepoint 实时捕获函数调用和系统调用事件流，获取**时间戳**、**线程上下文**、**调用栈**等运行时信息，以及函数执行耗时等动态指标。
3. **融合效果**：静态图谱提供拓扑骨架（有哪些可能的调用边），动态追踪提供时间轴（哪些边被实际执行、什么时候执行）。两者融合后，每条边都有了"是否被触发"和"触发时的上下文"的双重语义。

Q46: 为什么选择 eBPF 而不是内核模块来实现追踪？

答：

eBPF 相比内核模块有三大优势：

1. **安全性**：内核模块享有 root 权限，一旦有 bug 可能直接导致内核崩溃（BSOD/Oops）；eBPF 通过 Verifier 检查后在内核安全沙箱中运行，不会导致内核崩溃。
2. **可移植性**：不同内核版本的内核模块 API 差异很大，需要针对每个内核版本单独编译；eBPF 通过 CO-RE（Compile Once – Run Everywhere）机制实现了跨内核版本的可移植性。
3. **热更新能力**：eBPF 程序可以在运行时替换，无需卸载进程或重启系统；内核模块替换需要先卸载（`rmmmod`）再加载（`insmod`），过程中可能中断追踪。

对于需要长期稳定运行在生产推理服务器上的追踪系统，安全性和可移植性是硬需求，eBPF 是唯一选择。

Q47: 探针下沉策略的"保守挂载 + 增量采集"原则具体如何平衡完备性与高效性？

答：

"保守挂载"指仅对推理计算核心函数（`llama_decode`、`ggml_backend_sched_*`、`llama_synchronize`、`common_sampler_csampler`）挂载 uprobe，而非追踪所有函数，避免引入过多追踪点。

"增量采集"指参数级过滤（步骤四）：`read` 调用仅记录 `fd ≥ 3` 且字节数 `> 0`；`mmap` 仅记录 `≥ 1MB` 的分配。这些阈值来自推理引擎的实际行为特征（KV Cache 分配具有数百 MB 的特征），而非任意设定。

完备性保障：通过识别跨语言边界的关键咽喉节点（Go ↔ C++ 桥接点、推理计算入口点），确保跨语言调用链的主要流量不被遗漏。

高效性保障：通过保守挂载和参数过滤，将追踪开销控制在可接受范围内（TTFT 增量 `< 6.3%`，`bpfftrace` 进程 RSS 约 4MB）。

Q48: 缝合算法中, "滑动时间窗口"的时间窗口大小是如何确定的?

答:

时间窗口大小的确定基于推理引擎的运行时行为特征:

1. **基于经验测量**: 通过预实验测量跨语言调用的典型时间间隔 (例如, llama_decode 调用到其内部 mmap 的时间差通常在毫秒级)。
2. **基于推理阶段特征**: KV Cache 的 mmap 分配具有数百 MB 的特征, 而堆内存的微小扩展 (brk) 只有几 KB, 通过内存大小辅助判断窗口匹配的准确性。
3. **保守设置**: 窗口不能设得太大 (太大容易导致跨请求误匹配), 也不能太小 (太小会漏掉真正的关联事件)。实际实现中采用固定阈值 + 动态调整的策略。

缝合结果中 Confirmed 边 (直接调用栈确认) 的存在, 证明了时间窗口推断是有效的补充手段——即使窗口匹配不完美, 直接确认的边 (Confirmed) 也是可靠的。

Q49: 五层符号分类体系中的张量计算层 (f_2) 有哪些典型函数命名模式?

答:

张量计算层 (f_2) 负责神经网络推理的核心计算函数, 其典型命名模式包括:

- `ggml_*`: ggml 库的算子, 如 `ggml_mul_mat` (矩阵乘法)、`ggml_softmax`、`ggml_add`、`ggml_norm` 等。
- `llama_*` 且含 batch/sampler/token: 推理专用的批处理和采样函数, 如 `llama_decode`、`llama_batch_add`、`llama_sampler_sample`。
- `common_*`: 通用工具函数, 如 `common_sampler_csample`。
- `matmul`、`attention`、`softmax`、`layer_norm`、`embedding`: 深度学习领域的通用命名模式。

这些函数名具有强领域特征 (神经网络算子), 使得基于关键词匹配的自动分类成为可能——这也是五层分类体系得以实现的技术基础。

Q50: 缝合输出的 JSON 格式中, "layer_from"和"layer_to"字段的作用是什么?

答:

这两个字段记录了调用边跨越的五层分类层级:

- `"layer_from"`: 调用源节点所属的层次 (如 `"go_cgo_bridge"` $\rightarrow f_1$)。
- `"layer_to"`: 调用目标节点所属的层次 (如 `"llama_api"` $\rightarrow$ 默认 f 层或具体子类)。

这两个字段的作用是: 为每条缝合边赋予跨语言的语义身份。例如:

```
{ "from": "_cgo_*_Cfunc_llama_decode", "to": "llama_decode",  
  "layer_from": "go_cgo_bridge", "layer_to": "llama_api" }
```

这表示: f_1 (胶水层) 调用了 f 层 (计算层) $\rightarrow$ Go 通过 CGO 桥接调用了 llama.cpp 的推理函数。这使得分析人员无需理解具体的函数名, 只需理解层次语义, 就能判断调用关系的性质 (是跨语言调用、系统调用、还是内部计算调用)。

第 4 章 实现方案（25 题）

Q51: Ollama 的双进程模型（daemon + runner）如何影响你的追踪实现？

答：

Ollama 的 HTTP 服务进程（daemon）与模型运行进程（runner）为独立进程，两者通过 Unix Domain Socket（UDS）通信。这一架构特征对追踪实现产生了三个直接影响：

1. **追踪目标增加**：不能只追踪一个进程，需要同时追踪 daemon 和 runner 两个进程各自的 uprobe 和 tracepoint 事件。
2. **进程间关联困难**：daemon 接收 HTTP 请求，runner 执行推理计算，两者之间没有直接的函数调用关系，只能通过 UDS 通信的 socket 事件（read/write on UDS fd）建立跨进程关联。
3. **调用链缝合范围扩展**：缝合算法需要能够识别 UDS 通信事件，并将 daemon 侧的事件（HTTP 请求处理）与 runner 侧的事件（推理计算）关联为同一条端到端调用链。

实现中，通过追踪 runner 进程的 clone3 系统调用来识别 worker 线程创建（请求树根节点），并通过 socket 事件的时间相关性建立跨进程关联。

Q52: trace_unify.py 的三步清洗流程中，"事件标记识别"具体识别哪三种标记？

答：

三种关键标记：

1. **@UPROBE@** 标记：后跟函数名、时间戳、线程ID、耗时，标识一个 uprobe 事件的开始。
2. **@SYSCALL@** 标记：后跟系统调用名、进程ID、耗时及参数，标识一个系统调用事件。
3. **@END@** 标记：标识当前 uprobe 事件的结束，此时调用栈信息已完整，可以构造一条完整的调用事件。

这三种标记是 bpftrace 脚本在输出时主动注入的元信息，用于后续解析器识别事件边界。缺少这些标记，解析器无法区分原始文本中的哪一行属于哪条事件记录。

Q53: classify_func.py 如何处理 CGO 哈希后缀问题？请详细说明。

答：

classify_func.py 通过两层策略处理 CGO 哈希后缀问题：

第一层：C/C++ 符号（llama.cpp 层）：

- 对 llama.so、ggml.so 等动态库，使用 `nm -gC --defined-only` 提取全局导出符号。
- `-gC` 参数：`-g` 仅提取全局符号，`-C` 自动 demangle C++ mangled 符号（将 `_Z13llama_decodeP14llama_context` 还原为 `llama_decode(llama_context*)`）。
- 提取结果按前缀进入五层分类体系。

第二层：Go 运行时符号（Ollama Go 层）：

- 通过读取 `/proc/pid/maps` 获取内存映射区间，过滤出包含 Go runtime 代码的区间。
- 识别以下特征：
 - `runtime.` 前缀 → Go 运行时调度与内存管理（goroutine 调度、GC）。

- `runtime/cgo.` 前缀 → CGO 导出函数（跨越 Go ↔ C 边界的胶水节点）。
- 含 `_cgo_` 的函数名 → CGO 桥接层（`f1`）。关键：虽然哈希后缀（如 `_cgoexp_a1b2c3d4...`）无法 demangle，但其 `_cgo_` 前缀特征足以将其识别为胶水层函数，从而保证调用链的完整性。

Q54: `stitcher.py` 的三阶段置信度缝合算法能否处理跨进程的调用链？请说明。

答：

`stitcher.py` 当前的三阶段置信度缝合算法主要在单个 `runner` 进程内按 TID 构建请求树。对于跨进程（`daemon` ↔ `runner`）的调用链关联，实现中有一定的处理：

1. **clone3 作为根节点**：runner 进程通过 `clone3` 创建 worker 线程，该系统调用作为请求树的根节点，天然建立了进程创建的起点。
2. **UDS socket 事件关联**：daemon 向 runner 发送请求时，会触发 runner 侧的 `socket read` 事件，通过该事件的时间相关性，可以将 daemon 侧的 HTTP 请求处理与 runner 侧的推理计算关联起来。

但需要承认的是：当前实现对跨进程的关联是弱关联（基于时间窗口），而非强关联（基于调用栈直接确认）。daemon 和 runner 是两个独立的进程，`uprobe` 无法直接观测两者之间的 UDS 通信内容。这是当前实现的局限之一，在未来工作中通过增强 `socket` 事件分析来改进。

Q55: `uprobe` 的 `ustack(32)` 获取 32 层调用栈，如果调用栈深度超过 32 层会怎样？

答：

`ustack(32)` 表示最多获取 32 层调用栈。如果实际调用栈深度超过 32 层，超出部分会被截断，仅记录最内侧的 32 帧。

对于推理引擎来说，32 层已经非常充裕：

- Go HTTP Handler → API Router → Ollama Handler → CGO Bridge → llama.cpp → ggml → ... → CPU kernel
- 通常不超过 20 层

即使发生截断，最内侧的 32 帧也通常包含了最关键的跨语言边界节点（CGO 桥接函数和 llama.cpp 核心函数），不会影响跨语言调用链的还原。

Q56: 为什么 `mmap` 参数过滤选择 " $\geq 1\text{MB}$ " 作为阈值？这个阈值是如何确定的？

答：

" $\geq 1\text{MB}$ "阈值的确定基于推理引擎的实际内存分配行为：

1. **KV Cache 分配特征**：大模型推理的 KV Cache 需要缓存所有已生成 token 的 Key-Value 向量，其大小与模型参数量成正比。对于 TinyLlama-1.1B 和 Qwen2-0.5B，KV Cache 单次分配通常在数百 MB 到 1GB 之间。
2. **模型权重 mmap**：模型权重文件通过 `mmap` 映射到内存，单次映射大小通常在数 GB 级别。
3. **堆内存微调 (brk)**：普通的堆内存扩展每次只有几 KB 到几十 KB，与 KV Cache 的数百 MB 级别有数量级差异。

因此, "≥ 1MB"作为阈值可以有效区分推理相关的重大内存分配 (KV Cache、权重文件) 与无关的堆内存微调, 过滤掉大量噪声。

实验数据验证了这一阈值的有效性: TinyLlama 单次最大 mmap 600MB, Qwen 单次最大 mmap 768MB, 全部被保留; 而 brk/mprotect/madvise 等高频微调用被完全过滤。

Q57: futex_block 规则为什么设置 5 秒和 1 秒两个阈值? 它们分别对应什么告警级别?

答:

两个阈值的设置基于推理场景的 SLA 要求:

- **5 秒以上**: HIGH 级别告警。在交互式推理场景中, 5 秒以上的锁等待意味着用户会明显感知到响应延迟 (正常推理的首 token 时间通常在秒级以内), 这是一个严重的可用性风险。
- **1–5 秒**: MEDIUM 级别告警。1 秒以上的 futex 等待虽然不影响用户体验, 但表明存在性能瓶颈 (锁竞争激烈、I/O 阻塞), 需要关注但不至于立即告警。

实验数据显示: Qwen2-0.5B 的 23.83 秒追踪中, 共发现 30 条 HIGH 级别告警, 最长阻塞达 15.2 秒 (15,186ms), 平均阻塞 7.8 秒。这说明在 CPU-only 环境下, futex 锁竞争是一个普遍且严重的性能问题。

Q58: 为什么要对 Go 运行时符号的提取使用 `/proc/pid/maps` 而不是直接 `nm` 二进制文件?

答:

原因在于 Ollama 的编译和链接方式:

1. **Ollama 是静态链接的 Go 程序**: Go 程序默认静态链接整个运行时 (包括 goroutine 调度器、GC、网络栈等), 这意味着 Go 运行时符号被嵌入到主程序二进制文件中, 而不是作为独立的动态库。
2. **nm 二进制文件的不完整性**: 直接对 Ollama 可执行文件运行 `nm` 只能获取主进程的符号, 无法区分哪些符号属于 Go 运行时、哪些属于 Ollama 自己的代码 (符号名可能冲突)。
3. **/proc/pid/maps 的优势**: 运行中的 Ollama 进程通过 `/proc/pid/maps` 可以看到实际的内存映射区间, 包括 Go runtime 代码的实际加载地址和路径 (即使静态链接, 内核也会将 Go runtime 代码加载到特定虚拟地址区间)。通过过滤这些区间并结合符号名模式匹配 (`runtime.`、`runtime/cgo.`、`_cgo_`), 可以准确识别 Go 运行时符号。

Q59: analyze.py 的规则引擎中, 三条规则分别检测什么? 它们如何协同工作?

答:

三条规则:

1. **futex_block**: 检测 futex 等待时间超过阈值 (5s → HIGH, 1–5s → MEDIUM), 反映锁竞争或 I/O 阻塞。
2. **sensitive_path**: 检测对敏感路径 (`/etc/passwd`、`/root`、`/.ssh` 等) 的文件访问操作, 触发 HIGH 级别告警, 对应类似 Problama 的目录遍历攻击。
3. **mmap_size**: 检测 mmap 分配大小超过 1GB 或分配失败 (返回 -1), 触发 HIGH 级别告警, 反映异常的 KV Cache 分配。

协同工作方式：三条规则独立遍历事件序列，互不干扰。每条规则只关心自己类型的异常（futex_block 只看 futex 事件，sensitive_path 只看 openat/read 事件，mmap_size 只看 mmap 事件）。最终所有规则的告警按风险等级聚合输出——HIGH 级别告警优先展示，MEDIUM/LOW 级别告警作为补充上下文。

Q60: trace_unify.py 的状态机中，"追加 Event"操作是在 @END@ 标记处执行的，为什么？

答：

在 @END@ 标记处执行"追加 Event"操作的原因是：只有当 uprobe 函数返回时（@END@），该函数的完整调用上下文才全部就绪：

1. uprobe 记录了函数进入时刻（@start[tid] = nsecs）。
2. uretprobe（对应 @END@ 标记）计算了函数耗时（dur = (nsecs - @start[tid]) / 1000）。
3. 调用栈信息已完整：uprobe 的 ustack(32) 输出的所有栈帧在 @END@ 之前都已输出完毕。

因此，只有在 @END@ 处，才能将函数名、时间戳、线程ID、耗时、调用栈这五个字段组合为一条完整的调用事件，并追加到事件列表中。如果在 @UPROBE@ 处就追加事件，调用栈信息尚未收集完毕，事件记录不完整。

Q61: 为什么选择 bpfftrace 而不是直接写 C 语言 eBPF 程序？

答：

选择 bpfftrace 的原因是：

1. 开发效率：bpfftrace 提供了高级的 DSL（Domain Specific Language），一行脚本可以替代数十行 C 代码定义 map、附加探针、格式化输出，开发周期大幅缩短。
2. 快速原型验证：对于 PoC 阶段的研究，bpfftrace 的快速迭代能力是巨大优势——可以几分钟内写完并运行一个追踪脚本，立即看到数据。
3. 内置常用工具：bpfftrace 内置了 ustack(32)、@start[tid] 等常用追踪模式，减少了重复造轮子的工作。

但需要说明的是：bpfftrace 脚本最终需要转化为生产级的 C eBPF 程序（如使用 libbpf 和 CO-RE），以获得更好的性能和可移植性。这是未来工作（GPU 推理路径覆盖）中的计划之一。当前 PoC 阶段使用 bpfftrace 验证了方法可行性。

Q62: classify_func.py 的决策流程中，"llama_api"作为默认兜底层意味着什么？

答：

"llama_api"作为默认兜底层意味着：对于无法通过前缀关键词（llama_*、ggml_*、_cgo_）精确分类的 llama.cpp 函数，默认归入 llama_api 层。这是一个务实的兜底策略：

1. 保证分类覆盖率 100%：任何符号都能被分类，不存在"无法分类"的边缘情况。
2. 保留进一步分析的空间：虽然被归入默认层，但函数名本身（如 llama_whatever）仍然有分析价值，分析师可以进一步人工研判。
3. 符合分层设计的初衷：五层分类体系设计用于识别最有分析价值的层次（跨语言边界 → f₁、系统资源交互 → f₀、计算算子 → f₂、内存管理 → f₃、线程调度 → f₄），不属于这些核心层次的函数，默认归入通用 API 层是合理的。

Q63: 为什么在用户态用 Python 而不是 Go 或 C++ 实现 trace_unify.py、stitcher.py 等后处理脚本？

答：

选择 Python 的原因：

1. **数据清洗和解析的 I/O 密集特性**：trace_unify.py 的核心操作是逐行读取文本、过滤、解析 JSON——这些是字符串处理和文件 I/O 操作，Python 的标准库对文本处理非常高效（不需要手动内存管理）。
2. **开发和迭代速度**：Python 脚本的编写、调试和迭代速度远超 Go 或 C++，在 PoC 阶段尤为重要。
3. **JSON 处理**：Python 的 `json` 模块对 JSON 序列化/反序列化提供了原生支持，stitcher.py 的缝合结果以 JSON 输出，与后续分析工具（analyzer.py）的对接非常自然。
4. **性能足够的场景**：这些后处理脚本处理的是已经降噪后的数据（从数万行压缩到数千行），处理速度在毫秒级（Python 完全胜任），不是性能瓶颈。

Q64: analyzer.py 如何避免告警风暴（告警数量过多导致无法处理）？

答：

analyzer.py 通过以下机制避免告警风暴：

1. **独立规则 + 风险等级聚合**：每条规则独立遍历事件序列，独立生成告警，但最终按风险等级（HIGH/MEDIUM/LOW）聚合输出，分析师可以优先关注 HIGH 级别告警，忽略 MEDIUM/LOW。
2. **告警去重**：同一 TID 连续触发的同一规则（如连续多次 `futex` 阻塞），如果时间间隔过短，可以合并为一条带计数器的告警，而非逐条输出。
3. **采样窗口内的告警数量可控**：实验数据显示，23.83 秒采样期间共 30 条 HIGH 级别告警，全部由 `futex_block` 规则触发——告警数量在可处理范围内。
4. **上下文信息保留**：每条告警包含时间戳、TID、告警类型、风险等级和详细上下文，分析师可以快速判断告警的真实性，而不需要逐条研判原始数据。

Q65: 为什么只选择了 `llama_decode`、`ggml_backend_sched_graph_compute_async`、`llama_synchronize`、`common_sampler_csampler` 这四个函数挂载 uprobe，而不是更多？

答：

选择这四个函数是基于以下原则：

1. **覆盖推理全路径**：这四个函数覆盖了推理引擎从计算触发（`llama_decode`）到底层执行（`ggml_backend_sched_*`）再到输出生成（`common_sampler_csampler`）的完整路径，是跨语言边界最密集出现的关键节点。
2. **覆盖广度优先于深度**：与其在 20 个函数上各挂 1 个探针，不如在 4 个最关键的函数上确保探针可靠工作——这样可以完整覆盖这些函数内部的调用链（通过调用栈），间接覆盖更多下层函数。
3. **性能开销最小化**：探针数量与性能开销正相关。只挂 4 个探针对推理引擎的影响最小（TTFT 增量 < 6.3%），符合“高效性”的设计目标。

未来工作中计划增加更多探针以提升覆盖率（从当前的 Confirmed 5% → 更高）。

Q66: bpftrace 的 @start[tid] 为什么用线程ID (TID) 而不是进程ID (PID) 作为 key?

答:

因为推理引擎是多线程程序，使用 TID（而非 PID）作为 key 是必须的：

1. **每个线程有独立的调用栈**：Go 和 C++ 的调用栈是 per-thread 的，同一进程内不同线程的函数调用完全独立。`llama_decode` 在 TID=100 的线程中和 TID=101 的线程中是两个完全独立的调用事件，共享同一个 `@start` 记录会互相覆盖，导致错误的耗时计算。
2. **线程是调度和等待的单元**：futex 的等待和唤醒也是在特定 TID 上发生的，使用 TID 可以正确关联 futex wait 和对应的函数调用。
3. **Go 的 goroutine 与 OS 线程的映射**：Ollama 的 Go 层使用 goroutine（并发），但 goroutine 最终映射到 OS 线程（TID）执行，追踪 TID 可以捕获 Go 层的实际并发行为。

Q67: 为什么需要对 C++ 符号做 demangle 处理？不 demangle 会有什么影响？

答:

C++ 编译器对函数名进行名字重整（Name Mangling）是为了支持函数重载（overloading）——同名函数因参数类型不同而成为不同的符号。例如：

- `void foo(int)` → `_Z3fooi`
- `void foo(double)` → `_Z3food`

如果不 demangle，`_Z3fooi` 和 `_Z3food` 对于安全分析人员完全不可读，无法判断某个调用是调用了整数参数版本还是浮点数参数版本。对于缝合算法来说，虽然缝合主要依赖符号名的字符串匹配（匹配成功了就是成功了），但分类逻辑（如“含 batch/sampler/token → 张量计算层”）需要可读的函数名才能工作。如果不做 demangle，前缀匹配可能失败，导致分类不准确。

Q68: 在 stitcher.py 中，"阶段二：为每个节点记录 (tree_id, ts) 命中"的作用是什么？

答:

阶段二的核心作用是建立静态调用边与动态运行轨迹之间的关联矩阵：

1. **记录每个节点的出现位置**：对于同一个函数符号（如 `llama_decode`），可能在多棵不同的请求树（`tree_id`）中多次出现（每次 token 生成循环），每次出现的时间戳不同。这些命中记录（`(tree_id, ts)`）构成了该节点在运行时的完整轨迹。
2. **为阶段三的置信度判定提供数据基础**：阶段三遍历每条静态调用边时，需要检查该边的两端节点（A 和 B）是否在相同的请求树中同时出现。如果 A 和 B 在 `tree_id=1` 中分别在 `ts1` 和 `ts2` 被命中，且 `ts2 - ts1` 在合理范围内，则证明该调用关系被这次请求确认。
3. **支持跨请求的统计**：通过汇总所有 `(tree_id, ts)` 命中记录，可以统计每个函数符号在多次请求中的出现频率，用于判断哪些调用路径是最频繁的。

Q69: 为什么"过滤前后关键指标完全一致"这一验证是重要的?

答:

这一验证的重要性在于: 它证明了降噪操作不会引入假阴性(漏报)。

降噪的核心目标是去掉噪声(无语义价值的事件), 但如果过滤逻辑过于激进, 把有意义的事件也去掉了, 异常检测就会漏报。验证结果显示:

- `futex` 总次数: 过滤前 4,168 次, 过滤后 4,168 次 (0% 偏差)
- `futex` 总耗时: 过滤前 31,242ms, 过滤后 31,242ms (0% 偏差)
- `mmap` 总次数、最大 `mmap` 大小: 完全一致
- `openat` 总次数、有效 `uprobe` 事件: 完全一致

这意味着:

- 如果过滤前某次 `futex` 阻塞超过阈值, 过滤后该事件依然存在, 告警不会漏报。
- 如果过滤前某次 `mmap` 分配超过 1GB, 过滤后该事件依然存在, 告警不会漏报。
- 反之, 过滤前不存在的事件, 也不会过滤后凭空出现, 告警不会误报。

降噪操作只去掉了无语义价值的 `brk`、`mprotect`、`madvise` 等事件, 保留了所有与推理语义相关的关键系统调用。

Q70: 为什么将 Confirmed 边数 (≥ 2 棵树共同出现) 作为最可靠的置信度判定标准?

答:

" ≥ 2 棵不同的请求树共同出现"之所以是最可靠的置信度判定标准, 原因在于:

1. **排除偶然巧合**: 跨语言调用关系如果只在单次请求 (1 棵树) 中被同时观测到, 可能是该请求特有的行为 (如某种罕见错误处理路径), 不具有普适性。
2. **跨请求一致性**: 在 ≥ 2 棵请求树中同时出现的调用关系, 证明它是一种稳定的、跨请求的必然调用路径, 而非偶发行为。
3. **避免过拟合**: 如果把"1 棵树共同出现"也标记为 Confirmed, 会引入大量低置信度的边, 降低结果的可靠性。设置 ≥ 2 的阈值是一种保守策略, 宁可少报, 不可误报。

实验中 Confirmed 边全部落在 CGO 桥接层 (`Go` $\rightarrow$ `llama.cpp`), 且精确率 100% (无假阳性), 正是这一保守策略成功的证明。

Q71: `trace_unify.py` 输出 JSON 和折叠栈两种格式, 它们分别用于什么场景?

答:

两种格式服务于不同的下游工具:

1. **JSON 格式 (`trace_events.json`)**: 用于结构化的后续处理, 如缝合算法 (`stitcher.py`)、异常检测 (`analyzer.py`)。JSON 的键值对结构非常适合 Python 脚本读取和解析, 每条事件携带类型、时间戳、线程ID、函数名、调用栈等完整元信息。
2. **折叠栈格式 (`llama_folded.txt`)**: 用于火焰图 (Flame Graph) 可视化。折叠栈格式的每一行表示一条调用路径 (以分号分隔的函数名序列), 适合 Brendan Gregg 发明的火焰图工具直接渲染, 可以直观地展示推理引擎的函数调用热点和耗时分布。

这种双格式输出使得 HybriChain 的输出既可以用于程序化分析（JSON），也可以用于人工直观审视（火焰图）。

Q72: 为什么 Ollama 的 llama.cpp 层遵循一致的命名规范（如 llama_*、ggml_*）是五层分类体系实现的技术前提？

答：

五层分类体系的核心机制是基于函数名前缀和关键词匹配进行自动分类。例如：

- llama_* 且含 batch/sampler/token → 张量计算层（f₂）
- ggml_* → 张量计算层（f₂）
- 含 vocab/tokenize → 内存管理层（f₃）

这种分类策略能够work的前提是：函数命名必须具有规律性——同类功能的函数使用相同的前缀和命名模式。如果 llama.cpp 的函数命名混乱（如不同开发者在不同时间添加的函数使用完全不同的命名风格），基于前缀的分类就会失败，要么漏分类，要么误分类。

幸运的是，llama.cpp 作为社区维护多年的成熟项目，函数命名规范高度统一（遵循 POSIX/GNU 惯例），使得基于前缀匹配的自动分类完全可行。这是分类体系在 Ollama 上成功落地的关键工程前提。

Q73: 如何确保 stitcher.py 的缝合算法在极高并发场景下（如 1000 并发请求）仍然正确工作？

答：

当前实现基于以下假设：每个并发请求通过独立的 TID（worker 线程）执行，缝合算法按 TID 构建独立的请求树。但在极高并发场景下（1000+ 并发），可能面临：

1. **线程池复用**：llama.cpp 使用线程池，worker 线程可能被多个请求复用（通过取走/归还任务的方式），导致同一 TID 在不同时间服务于不同请求。如果不加以区分，缝合算法会把不同请求的事件混在同一棵请求树中，导致调用链错误关联。
2. **当前解决方案**：stitcher.py 使用 clone3 系统调用作为请求树的根节点（标记新请求的创建），通过 clone3 的时间戳来区分同一 TID 上不同请求的生命周期。但 clone3 的识别和去噪在高并发下也是一个挑战。

这是当前实现的一个局限，在未来工作中通过引入请求 ID（通过 UDS 通信中的元数据）来增强跨请求的区分能力。

Q74: 为什么异常检测规则的阈值（如 futex > 1s、mmap > 1GB）是硬编码的，而不是自适应的？

答：

阈值硬编码的原因：

1. **简单性和可解释性**：硬编码阈值使得告警逻辑完全透明可解释——"futex 阻塞超过 5 秒 → HIGH"，分析师可以立即理解告警含义，不需要理解复杂的自适应算法。
2. **基于领域知识设定**：这些阈值基于推理引擎的实际运行特征（见 Q57 的分析），而非数据驱动的人工智能模型，在小样本场景下比自适应方法更可靠。
3. **安全审计的取证标准**：在安全审计场景中，告警规则需要可复现、可解释——同样的输入必须产生同样的告警，自适应阈值可能导致不同时间对同一行为给出不同判断，不符合取证要求。

未来工作中计划引入基于机器学习的自适应异常检测（anomaly detection），但那是规则引擎的补充而非替代——规则引擎提供确定性告警，ML 模型提供概率性异常预警。

Q75: analyze.py 输出的告警 JSON 中，每个字段的含义是什么？请解释

`{"tid":45633,"type":"futex_block","risk":"HIGH","dur_ms":15186.0}`。

答：

字段含义：

- `"tid": 45633`：发生异常的线程 ID（Thread ID），通过 `/proc/<pid>/task/<tid>` 可以进一步查看该线程的调用栈上下文。
- `"type": "futex_block"`：告警类型，标识这是 futex 阻塞异常（对应 analyzer.py 中的 futex_block 规则）。
- `"risk": "HIGH"`：风险等级，本次告警是 HIGH 级别（因为 `dur_ms = 15,186ms > 5,000ms`）。
- `"dur_ms": 15186.0`：异常的持续时间（毫秒），本次 futex 阻塞持续了约 15.2 秒——这意味着该线程持有锁或等待锁的时间长达 15.2 秒，是极其严重的锁竞争事件，可能导致该推理请求的响应延迟增加十余秒。

第 5 章 实验评估（20 题）

Q76: Ground Truth 基准数据集是如何构建的？20 条调用路径是如何得出的？

答：

Ground Truth 基准数据集通过源码级静态走读构建：

1. 对 llama.cpp 源码中的所有函数调用语句（read/write 调用）进行穷尽式扫描。
2. 提取所有在正常推理流程中**必然发生**的跨语言调用路径（即不是错误处理路径、不是条件分支中的罕见路径）。
3. 经过人工筛选，得到 20 条覆盖推理全流程的关键调用路径。

这 20 条路径分为五类：

- CGO 桥接链路（5 条）：`_cgo_*_Cfunc_llama_*` → `llama_*`
 - 采样链路（3 条）：`llama_sampler_sample` ↔ `common_sampler_sample`
 - 调度计算链路（5 条）：`llama_decode` → `ggml_backend_sched_*`
 - 模型加载链路（4 条）：`llama_model_load_from_file` → `ggml_*`
 - 批量处理链路（3 条）：`llama_batch_add` → `common_batch_add`
-

Q77: 召回率仅 5% (3/20)，这个结果是否说明缝合算法本身存在严重缺陷？

答：

（关键问题，需要有力回应）召回率 5% 不是缝合算法本身的缺陷，而是探针覆盖率的直接体现，原因如下：

1. **Ground Truth 是"所有可能路径"**，但实际推理只走子集：20 条 Ground Truth 路径是静态源码分析得出的"所有可能调用的路径"，但一次具体推理请求（响应一个用户 query）只走其中子集。
2. **uprobe 只挂了 4 个函数**：当前仅在 llama_decode、ggml_backend_sched* 等 4 个函数上挂载 uprobe，这 4 个函数内部调用的子函数（如 llama_decode 内部调用的 ggml* 函数）在源码静态图中存在，但动态追踪中因为没有独立探针，只能标记为 Unknown。
3. **Confirmed 边恰好是"最有价值"的 3 条**：全部位于跨语言边界（Go ↔ C++），证明核心问题（跨语言调用链断裂）已被成功解决。

关键结论：召回率低是探针覆盖率的问题，不是缝合算法的问题。缝合算法在 Confirmed 边上的精确率是 100%（无假阳性），含 Inferred 边召回率达到 40%，证明算法本身是有效的。增加探针覆盖率（增加更多 uprobe 目标函数）可以线性提升召回率。

Q78: TTFT 从 65ms 到 66ms（增加 1.5%）和从 64ms 到 68ms（增加 6.3%），两个模型的增幅差异为什么这么大？

答：

两个模型的增幅差异（1.5% vs 6.3%）主要原因：

1. **模型规模差异**：TinyLlama-1.1B（1.1B 参数）和 Qwen2-0.5B（0.5B 参数）的计算复杂度不同，Qwen2 虽然参数更少，但可能有不同的推理配置（batch size、上下文长度等）导致单次推理的计算量不同。
2. **uprobe 挂载点触发频率**：每个 token 生成循环中，llama_decode 都会被调用。Qwen2-0.5B 的每次 llama_decode 调用可能触发更多的 uprobe 回调（取决于模型结构和批处理配置），累积的性能开销更高。
3. **测量误差**：6.3% 的增量（4ms）在毫秒量级上可能包含测量误差（系统负载波动、网络延迟等外部因素），应将其理解为"同一量级"而非"精确的 6.3%"。

无论如何，6.3% 的增量都在可接受范围内，不会显著影响推理引擎的实时响应能力。bpftrace 进程 RSS 仅 4MB，相比推理引擎本身数 GB 的内存占用可以忽略不计。

Q79: 23.83 秒的采样时长如何保证能捕获到有代表性的推理行为？

答：

23.83 秒采样时长的代表性问题需要客观承认：

1. **覆盖了推理全生命周期**：在这个时间段内，覆盖了模型加载（mmap 大量权重文件）、模型初始化（futex 线程创建）、推理执行（llama_decode 循环，Qwen 生成 10 个 token）三个完整阶段。
2. **PoC 阶段的合理切片**：这是用于验证框架可行性（Feasibility）的 Proof of Concept 切片，而非生产级基准测试。目的是证明"方法有效"，而非"覆盖所有场景"。
3. **长尾路径的局限**：采样窗口外的长尾路径（如首次加载后不再触发的初始化路径、边缘错误处理路径）确实未被捕获，这是采样窗口的自然限制。

Q80: 为什么 Qwen2-0.5B 生成了 10 个 token 而 TinyLlama-1.1B 生成了 190 个 token？这对实验结果有什么影响？

答：

生成 token 数量的差异可能由以下因素导致：

1. **测试 prompt 不同**：两个模型接收的输入 prompt 可能不同，导致生成长度不同。
2. **采样参数不同**：temperature、top-p、max_tokens 等采样参数设置不同。
3. **模型特性差异**：不同模型的典型输出长度不同。

对实验结果的影响：

- TinyLlama 的 llama_decode 循环次数更多（119 次），系统调用（futex、mmap）的总触发次数更多，因此 TinyLlama 的原始事件数（25,329 行）大于 Qwen（13,670 行）。
- 但压缩比（2.8x vs 2.1x）和处理时间（6.8ms vs 4.4ms）的差异也反映了这一点——更多事件意味着更多的 brk/mprotect 噪声被过滤。
- 两个模型的结果差异也证明了 HybriChain 对不同推理场景的适应性。

Q81: 为什么"并发阻塞峰值（9–10 秒级）"表明问题具有"系统性"而非"偶发性"？

答：

"系统性"与"偶发性"的区别在于：

- **偶发性**：单次事件，与特定请求或特定时间点相关，不会在重复实验中复现。
- **系统性**：结构性问题，在相同条件下可重复触发，涉及多个独立组件。

在实验中，第二 token 生成周期的 9–10 秒区间内，10 条告警几乎同时触发，TID 分布在两组 worker 线程中（TinyLlama runner 的 TID 41052–41123 与 Qwen runner 的 TID 45624–45633），两组来自不同模型的 worker 线程同时出现阻塞，说明这是 llama.cpp 在 CPU-only 环境下多线程同步机制的结构性瓶颈——无论输入什么模型、什么请求，只要并发请求存在，futex 锁竞争就会发生。这是"系统性"而非"偶发性"的有力证据。

Q82: 30 条 HIGH 级别告警全部由 futex_block 规则触发，为什么 sensitive_path 和 mmap_size 规则没有触发告警？

答：

两个原因：

1. **sensitive_path 未触发**：说明在正常的模型推理过程中，没有发生对敏感路径（如 /etc/passwd、/root、/.ssh）的文件访问。这是预期的——正常的 Ollama 推理不需要访问这些敏感文件。如果 sensitive_path 规则触发了，说明要么是 Problama 类型的攻击正在发生，要么是 Ollama 自身的异常行为——两种情况都需要告警。
2. **mmap_size 未触发**：说明正常推理过程中没有发生超过 1GB 的单次 mmap 分配（虽然总 KV Cache 分配达到数 GB，但可能分多次完成），也没有 mmap 分配失败。如果该规则触发，说明发生了异常的内存分配行为。

sensitive_path 和 mmap_size 规则没有触发告警，恰恰说明本次追踪是在**正常推理条件**下进行的，验证了 HybriChain 在正常场景下不会产生误报。

Q83: "过滤前后关键指标完全一致"的验证中，过滤掉了哪些无用事件？它们占原始数据的多大比例？

答：

过滤掉的高频内存管理调用：brk（堆内存微调）、mprotect（内存保护调整）、madvise（内存管理建议）、munmap（内存释放）、clock_gettime（时间查询）等。

以 TinyLlama-1.1B 为例：

- 原始事件：25,329 行
- 有效事件（过滤后）：8,891 行
- 被过滤事件：16,438 行（占原始的 **65%**）
- 其中 brk/mprotect/madvise/munmap/clock_gettime 合计约 16,438 条

这些被过滤的调用虽然数量巨大（占原始数据的 65%），但每个调用只涉及几 KB 到几十 KB 的内存调整，与推理语义无关（不涉及跨语言调用链、不会触发安全告警），过滤后不影响 futex、mmap、openat、uprobe 等关键事件的统计。

Q84: 为什么 TinyLlama 的原始事件数（25,329 行）是 Qwen（13,670 行）的近两倍，但压缩比反而更高（2.8x vs 2.1x）？

答：

这个现象看似矛盾，但逻辑上是一致的：

1. **TinyLlama 原始事件更多**（25,329 vs 13,670）：因为 TinyLlama 生成了 190 个 token，推理循环次数更多（119 次 llama_decode），触发的系统调用也更多。
 2. **压缩比更高**（2.8x vs 2.1x）：说明 TinyLlama 的噪声事件（brk/mprotect/madvise/munmap/clock_gettime）占比更高——因为 119 次推理循环中，每一次循环都触发大量堆内存微调调用（brk），这些高频调用在总事件数中占比更大，过滤掉后压缩比更高。
 3. **Qwen 的有效事件绝对数更多**（6,373 vs 8,891 指的是 TinyLlama 的有效事件更多）：虽然 Qwen 原始事件更少，但有效事件数 Qwen 是 6,373，TinyLlama 是 8,891——这里需要确认原文数据的一致性，答辩时注意准确引用。
-

Q85: "Confirmed 边全部落在 CGO 桥接层"这一发现有何重要意义？

答：

这一发现有两层重要意义：

1. **证明了跨语言边界追踪的核心问题已被解决**：CGO 桥接层（Go ↔ C++）正是跨语言调用链最脆弱、最难追踪的环节——Go 编译器生成的哈希后缀使得符号不可读，CGO ABI 与 Go 运行时的差异使得传统追踪工具失效。HybriChain 的 uprobe 在 llama.cpp 核心函数上成功捕获了来自 CGO 桥接的调用栈，证明跨语言边界可以被准确追踪。
2. **证明了 Confirmed 置信度的可靠性**：3 条 Confirmed 边全部是真实可靠的调用关系，精确率 100%——说明基于“≥2 棵请求树共同出现”的置信度判定策略是有效的，没有假阳性。

这一发现是全文最重要的实验结论之一。

Q86: 为什么说"futex 调用占据绝对主导地位"这一发现与第二章的架构分析形成了"完美的闭环验证"?

答:

第二章（相关技术综述）指出 Ollama 依赖大块 mmap 加载模型权重（数百 MB 到数 GB）和大量 futex 实现多线程同步（Go goroutine 调度器通过 futex 实现等待/唤醒）。第五章的实验数据验证了这一点：

- TinyLlama 全生命周期约 4,168 次 futex 调用，Qwen 约 1,131 次——说明并发推理中线程同步非常频繁。
- llama_decode 循环中每次迭代都触发 futex（等待前序操作完成），而 llama_decode 被调用 119 次（TinyLlama）和 72 次（Qwen），加上线程池维护的 futex，总数完全吻合。
- mmap 调用中单次最大分配 600MB（TinyLlama）和 768MB（Qwen），与模型权重文件大小直接相关。

实验数据与第二章的理论分析一一对应，形成了"理论预测 → 实验验证"的闭环，证明了 HybriChain 缝合算法的正确性——它确实追踪到了推理引擎在第二章中描述的那些架构行为。

Q87: 推理阶段自动划分为"进程初始化 → 进程 fork → 模型加载 → 推理执行"，这一划分完全基于缝合序列的语义特征，具体依据是什么？

答:

四个阶段的划分依据缝合序列中的标志性系统调用和函数事件：

1. **进程初始化**：在 clone3（创建 worker 线程）之前，推理引擎需要初始化环境。这个阶段可能表现为一些 openat/read/close 系统调用（读取配置文件、环境变量等）。
2. **进程 fork（worker 线程创建）**：以 clone3 系统调用为标志——Ollama daemon 通过 clone3 创建 runner 进程/线程，这是推理请求生命周期的起点。
3. **模型加载**：以 mmap 系统调用为标志——mmap 映射模型权重文件（只读，数百 MB 到数 GB）和 KV Cache 内存区域（读写，数百 MB）。这一阶段的核心行为是大块 mmap。
4. **推理执行**：以 llama_decode 的循环调用为标志——llama_decode 被重复调用，每次生成一个 token，同时伴随 futex（等待/同步）、mmap（KV Cache 扩展）等系统调用。

完全基于事件序列的语义特征自动完成，无需人工标注，这证明了缝合算法的有效性——它能够从原始事件流中识别出具有明确语义的高层行为模式。

Q88: "平均阻塞时长 7.8 秒"和"最长阻塞 15.2 秒"，这两个数字对推理系统的可用性意味着什么？

答:

- **平均阻塞 7.8 秒**：意味着在 23.83 秒的采样窗口内，30 次 futex 阻塞的平均等待时间为 7.8 秒。这是一个极其夸张的数字——正常情况下，一次 token 生成的总时间应该在秒级以内，而仅仅是锁等待就占据了 7.8 秒。这意味着在 CPU-only 环境下，llama.cpp 的多线程同步机制是严重的性能瓶颈。
- **最长阻塞 15.2 秒**：意味着在一次 token 生成循环中，某线程持锁或等待锁的时间长达 15.2 秒。对于交互式推理场景，这可能导致用户感受到的响应延迟从理论最优值（毫秒到秒级）增加到 15 秒以上，严重影响用户体验。

- **阻塞总时长 > 244 秒**：说明在 23.83 秒的采样窗口中，所有线程的 `futex` 阻塞总时长超过 244 秒——平均每秒真实时间对应约 10 秒的阻塞时间，说明 CPU 资源大量浪费在锁等待上，而非实际推理计算。

这揭示了 AI 推理服务在并发场景下的 **DoS 脆弱点**：攻击者可以通过构造大量并发推理请求，使多线程锁竞争加剧，导致正常用户的推理请求也被阻塞，产生类似 DoS 的效果。

Q89: 为什么实验只用了两个模型（TinyLlama-1.1B 和 Qwen2-0.5B），没有覆盖更大规模的模型？

答：

选择这两个模型的原因：

1. **硬件资源限制**：更大规模的模型（如 Llama-7B、Llama-13B）需要数十 GB 的内存和更高的算力，在 CPU-only 环境下可能无法运行，或运行时间过长导致实验周期不可控。
2. **PoC 阶段的聚焦**：本研究的重点是验证“跨语言调用链追踪方法”的有效性，而非评估不同规模模型的性能差异。方法论对模型规模无关——无论是 0.5B 还是 70B，只要使用 Ollama（Go-CGO-C++ 架构），HybriChain 的追踪方法都适用。
3. **模型规模的扩展性已在架构设计中体现**：五层符号分类体系基于函数名而非模型参数，因此天然支持不同规模的 llama.cpp 模型。

答辩时老师可能追问：应坦诚说明这是资源约束下的选择，并指出 HybriChain 的方法论设计不受模型规模限制，未来计划在更大规模模型上验证（GPU 环境下）。

Q90: 过滤前后 `futex` 总次数完全一致（4,168 次），说明 `futex` 调用全部被保留，这是否意味着 `futex` 调用不属于“噪声”？

答：

完全正确。`futex` 调用是推理引擎中最有分析价值的系统调用之一，原因：

1. **线程同步是性能瓶颈**：第 5.4 节的异常告警数据表明，`futex` 阻塞是推理引擎最严重的可用性风险（最长阻塞 15.2 秒），是性能优化的关键抓手。
2. **反映并发调度行为**：`futex` 调用的高频出现（4,168 次，占总系统调用数的重要比例）反映了推理引擎的并发调度策略和线程池管理行为，是理解推理性能的重要窗口。
3. **区分推理阶段**：通过分析 `futex` 调用的时间分布，可以区分推理的不同阶段（初始化期、推理期、同步期）。

因此，`futex` 调用不应被过滤，而是异常检测的核心对象——`analyzer.py` 的 `futex_block` 规则正是基于它来检测锁竞争风险。

Q91: 为什么推理引擎的内存占用“对数坐标”是重要的？用线性坐标会有什么影响？

答：

用对数坐标的原因：

1. **数量级跨越大**：推理引擎的内存占用从数十 MB（进程启动）到数 GB（模型权重 `mmap`）再到数十 GB（KV Cache），跨越 3 个数量级（ $10^4 \rightarrow 10^{10}$ 字节）。

2. **线性坐标无法展示细节**：如果用线性坐标，数十 MB 的初始占用在数 GB 的图表中被压缩到接近零，初始阶段的内存变化完全不可见。对数坐标将不同数量级的数据压缩到同一可视化空间，使各个阶段的内存变化都能清晰展示。
3. **展示模型加载的突变**：对数坐标下，mmap 映射权重文件时的内存"台阶式"跃升清晰可见（从数十 MB 跃升到数 GB），这是推理引擎内存管理的典型特征。

Q92: 实验数据中 "静态调用图：4832 节点 / 52 边", 4832 个节点是否过多？与实际追踪的 4 个探针函数是否矛盾？

答：

4832 个节点来自静态符号提取（`nm` 工具对 Ollama 二进制文件的全局符号提取），包括：

- Go 运行时所有函数（`runtime.`、`net.`、`os.*` 等数千个函数）
- Ollama 自身所有 Go 函数（API Handler、路由、中间件等）
- `llama.cpp` 所有 C++ 函数（数千个）

这并不与"只挂 4 个探针"矛盾：

- **4832 节点是静态的"可能调用的所有函数"**：这些是 Ollama 进程中所有可能出现的函数符号，为缝合算法提供符号映射的候选集。
- **4 个探针是动态的"实际追踪的入口点"**：`uprobe` 挂在 4 个核心函数上，通过调用栈可以间接观测到这些核心函数调用的下层函数。
- **52 条静态边**：是从 `llama.cpp` 源码静态分析得出的跨语言调用边（并非所有 4832 个节点之间的边）。

两者结合，静态图谱提供节点集和边集，动态追踪验证哪些边被实际触发，缝合算法将两者融合。

Q93: 为什么说"bpfttrace 追踪进程的内存占用约 4MB"这一数据对评估系统的实用性至关重要？

答：

4MB 的内存占用对系统实用性的意义：

1. **在生产环境中可忽略不计**：推理引擎本身占用数 GB 内存（模型权重 + KV Cache），4MB 相比之下是 0.1% 量级，完全不会导致 OOM 或资源耗尽。
2. **可以在生产环境中长期运行**：如果 HybriChain 的追踪进程本身消耗大量内存（如数百 MB），在内存受限的边缘设备上就无法部署。4MB 的占用意味着它可以在任何标准推理服务器上长期运行，无需专门的资源预留。
3. **证明 eBPF 方案的低开销特性**：4MB 主要用于存储 per-event 数据结构（哈希表、环形缓冲区）和 BPF map，相比 Valgrind 的影子内存（需要与目标进程同等规模的额外内存），eBPF 的资源消耗几乎可以忽略。

Q94: 实验的"过滤正确性验证"中，过滤前后 "uprobe 事件完全一致"说明了什么？

答：

过滤前后 uprobe 事件完全一致（6,247 个）说明了：

- uprobe 事件不参与类型过滤：** 五步降噪流水线的步骤三（类型过滤）只针对系统调用（syscall），不针对 uprobe 事件——因为 uprobe 是用户态函数调用，不属于内核系统调用类型白名单过滤的范围。
- 降噪操作不会丢失函数级观测数据：** 所有 llama_decode、ggml_backend_sched_*、llama_synchronize、common_sampler_csample 的 uprobe 事件都被完整保留，这是缝合算法能够工作的数据基础。
- 降噪只作用于内核态系统调用：** 降噪去掉的是 brk、mprotect、madvise、munmap、clock_gettime 等无语义价值的内核系统调用，用户态的函数调用观测（uprobe）完全不受影响。

Q95: "精确率达到全部确认"和"含推断召回率达到 40%"，这两个数字如何向非技术人员解释其含义？

答：

向非技术人员解释：

- 精确率 100%（全部确认）：** 可以理解为"HybriChain 说有的调用链，都是真实存在的调用链，没有瞎编的"。精确率是安全审计的硬性要求——如果精确率低，意味着大量假阳性告警，分析师会被淹没在误报中，系统不可用。
- 召回率 40%（含推断）：** 可以理解为"在所有真实存在的调用链中，HybriChain 找到了 40%"。召回率受限于探针覆盖率——当前只追踪了 4 个核心函数，如果追踪更多函数，召回率会更高。

打个比方：HybriChain 像一个"高精度探测器"——它不会误报（精确率高），但还有一部分真实调用链没被发现（召回率有待提升），原因是探测器的"探头"还不够多（探针覆盖率有限），未来增加更多探头就能发现更多。

第 6 章 总结与展望（5 题）

Q96: 论文工作总结中提到"五级锚点匹配方法"，但论文正文（第三、四章）说的是"五层符号分类体系"，两者是否一致？

答：

（坦诚回答，注意一致性）"五级锚点匹配方法"（第6章工作总结）与"五层符号分类体系"（第3、4章）是同一技术的不同表述：

- 五层符号分类体系（f₀—f₄）：** 是核心数据结构，将函数符号分类到 5 个语义层次（系统边界 f₀、运行时胶水 f₁、张量计算 f₂、内存管理 f₃、调度协调 f₄）。
- 五级锚点匹配方法：** 是应用层面的描述——在缝合算法中，五层分类为每条跨语言边提供了"锚点"（源节点和目标节点的层次标签），通过匹配这些锚点可以确定调用关系的语义。

两者是同一个技术方案在不同视角下的描述：分类体系是数据结构，锚点匹配是使用该数据结构的算法。如果答辩老师追问，应统一使用"五层符号分类体系"的表述，避免混淆。

Q97: 论文提到"含 inferred 边召回率 70%"（第6章工作总结），但第5章实验数据显示是 40%（12/20），哪个数字是正确的？

答：

（关键问题，需要仔细核对）第6章工作总结中的"70%"与第5章实验数据中的"40%（含 Inferred 召回率 12/20 = 60%，原文写 40%）"存在不一致：

- 第5章（实验数据）：Confirmed 3/20（5%），含 Inferred 12/20（60%，原文写 40% 可能为笔误或两处数据来源不同）。
- 第6章：含 Inferred 边召回率 70%。

建议答辩时的应对：

1. 承认两处数据可能存在不一致，需要以实验章节（第5章）的原始数据为准。
2. 如果第6章的 70% 是更新的数据，应说明是补充实验的结果。
3. 答辩前应与导师核对所有数据的一致性，确保全文数据统一。

Q98: 召回率从 10%（Confirmed）提升到 70%（含 Inferred），提升幅度很大，但 Inferred 边是否可靠？

答：

Inferred 边的可靠性需要辩证看待：

- **优势**：Inferred 边通过时间窗口启发式推断，能够补充大量 Confirmed 边无法覆盖的调用关系（如 llama.cpp 内部函数之间的调用），将召回率从 5—10% 提升到 40—70%，是缝合算法的重要补充。
- **局限**：Inferred 边基于时间窗口匹配，可能存在跨请求误匹配——如果两个不同请求的事件在时间上接近，可能被错误地缝合在一起。
- **置信度区分的价值**：正是因为 Inferred 不如 Confirmed 可靠，HybriChain 才设计了置信度标签体系，让分析人员能够区分哪些边是确定的（Confirmed）、哪些是推断的（Inferred），从而做出可靠的安全判断。

在实际安全审计中，Inferred 边可以作为"可疑线索"引导人工深入分析，但不应作为唯一的取证依据——这是系统设计的刻意设计，而非缺陷。

Q99: GPU 推理路径覆盖作为未来工作的第一项，CUDA Profiling Tools Interface（CUPTI）具体如何集成到 HybriChain？

答：

CUPTI 集成到 HybriChain 的具体计划：

1. **GPU 指标采集层**：通过 `nvidia-smi dmon` 和 `nvidia-container-toolkit` 采集 GPU 利用率和显存使用数据，作为 eBPF 内核追踪数据的补充。
2. **CUDA API 追踪**：通过 CUPTI 追踪 CUDA API 调用（`cuMemcpy`、`cuMemAlloc`、`cuLaunchKernel` 等），建立从 Python `torch.nn.Module.forward` 到 CUDA kernel 的完整调用链。
3. **CPU-GPU 协同分析**：将 CPU 侧的 eBPF 追踪数据（Go HTTP Handler → CGO → llama.cpp → CPU 计算）与 GPU 侧的 CUPTI 追踪数据（CUDA API → kernel 执行 → 显存操作）通过时间戳对齐融合，形成覆盖 CPU-GPU 全栈的端到端调用链。

4. 容器环境适配：Ollama 在容器中运行时，需要处理容器 GPU 指标（通过 cgroup 和 nvidia-container-toolkit）与主机侧 eBPF 追踪数据的关联问题。

当前实现专注于 CPU 推理路径，GPU 路径是下一步工作方向。

Q100：你认为 HybriChain 最核心的贡献是什么？用一句话总结。

答：
HybriChain 最核心的贡献是：首次实现了对 Go-CGO-C++ 三层混合编程推理引擎的端到端跨语言调用链追踪，在不影响推理性能的前提下（TTFT 增量 < 6.3%），从高层 API 请求一路追踪到内核系统调用，填补了开源 AI 软件在多语言架构下的可观测性与安全审计空白，为类似 Problama 的 API 安全漏洞提供了实时溯源的技术手段。

附录：高频压力问题清单

以下问题在答辩中出现的概率最高，需要重点准备：

优先级	问题	核心挑战
★★★★	召回率为什么这么低？算法有问题吗？	证明是探针覆盖率而非算法缺陷
★★★★	为什么只追踪 4 个函数？漏掉了怎么办？	解释"保守挂载"原则和扩展计划
★★★★	23.83 秒采样够吗？结论有代表性吗？	承认 PoC 局限，说明生产部署计划
★★★★	第6章的 70% 与第5章的 40% 数据不一致？	核对数据，准备统一版本
★★★☆☆	GPU 场景怎么办？	诚实承认局限，展示未来计划
★★★☆☆	能防御真实的 Problama 攻击吗？	从规则设计和缝合链路两个角度回答
★★★☆☆	为什么不用 X 工具（strace/DynamoRIO...）？	第1章和第2章已有论述，简洁复述
★★☆☆☆	文献年份（2026）是怎么回事？	坦诚说明，承诺核实
★★☆☆☆	五级锚点和五层分类是什么关系？	统一表述为"五层符号分类体系"
★★☆☆☆	4832 节点 vs 4 个探针，矛盾吗？	静态覆盖 vs 动态精准的互补关系